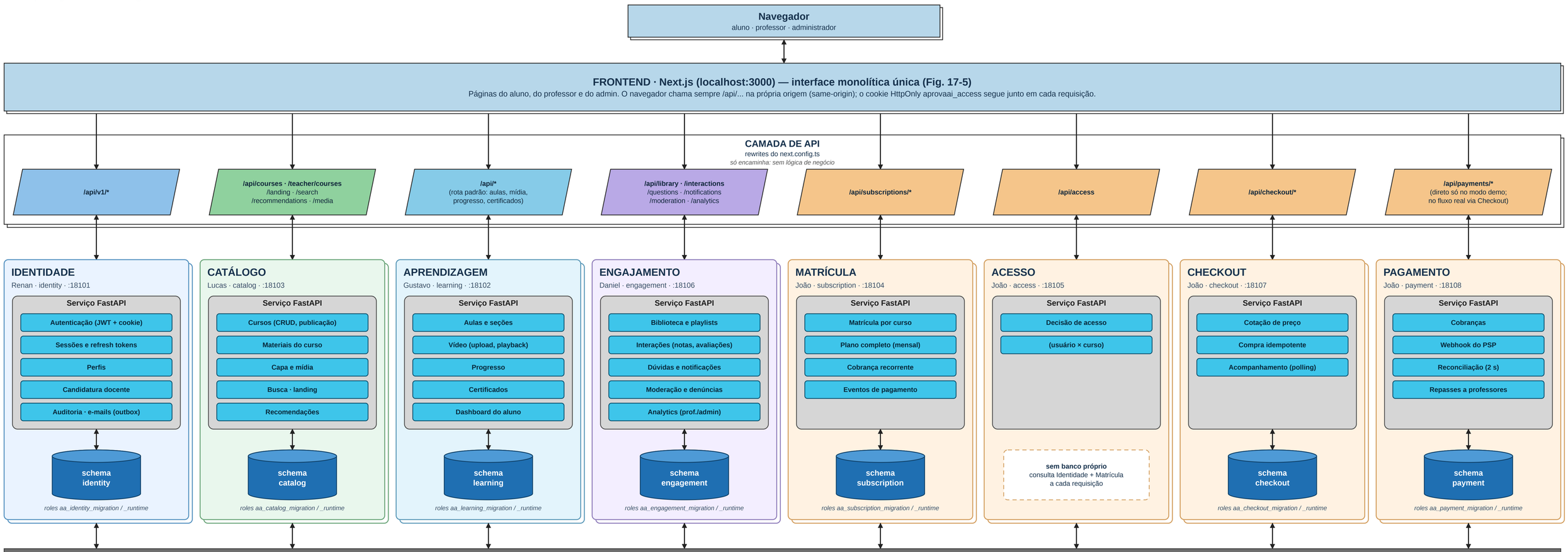
AprovaAí — Modelo arquitetônico da integração

Estilo: microsserviços (Richards & Ford, Fundamentals of Software Architecture, cap. 17). Cada serviço é um contexto delimitado com dados próprios; todos se comunicam por HTTP/REST.

① Topologia — do navegador ao banco



② Quem chama quem — dependências HTTP lidas no código

Linha = quem chama · coluna = quem responde · ★ = escrita interna com token de serviço

chamador ↓ responde →	Identidade	Catálogo	Aprendiz.	Matrícula	Acesso	Pagamento	PSP (mock)	Metering / Conteúdo
Catálogo	auth/me profiles/me		lesson-summaries		GET /access			
Aprendizagem	auth/me profiles/me	courses course summary		subscriptions/me	GET /access			
Engajamento	auth/me · profiles POST audit ★	course summary PUT moderation ★	lesson summary PUT moderation ★		GET /access			
Matrícula	auth/me	course summary				POST /payments ★ (renovação mensal)		
Acesso	auth/me			subscriptions/me				
Checkout	auth/me	course summary (preço)		subscriptions/me		POST · GET /payments ★		
Pagamento				POST events/ payment ★			charges transfers	usage/summary items/owners
PSP (mock)						webhook (assíncrono)		
Conteúdo (mock)					GET /access			

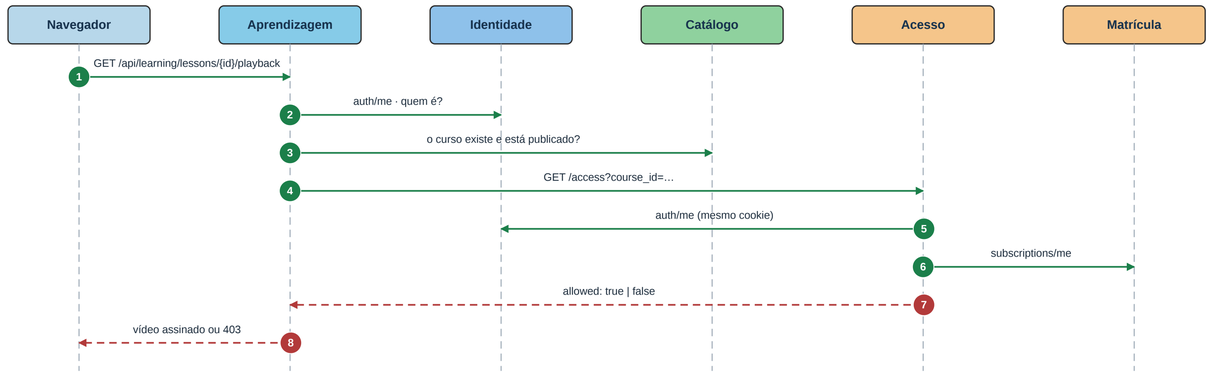
Como ler o diagrama

- Requisição HTTP síncrona: quem chama espera a resposta.
- Assíncrono ou resposta posterior: webhook do PSP, retorno, polling do navegador.
- Ligação de mão dupla: camada de API ↔ serviço, serviço ↔ seu schema.
- ★ Escrita entre serviços: além do usuário, exige X-Service-Token igual ao INTERNAL_SERVICE_TOKEN.
- Cores = contextos/equipes, as mesmas do modelo ER.

Regra de ouro: um serviço nunca lê o banco de outro. Dado alheio vem sempre pela API do dono (contexto delimitado, cap. 17).

③ Fluxo: assistir uma aula — coreografia

Não há mediador: cada serviço consulta diretamente quem precisa (Fig. 17-7).



④ Fluxo: comprar um curso — orquestração

Checkout é o dono do fluxo (Fig. 17-10); pagamento assíncrono e idempotente, reconciliado a cada 2 s.

